

Deep Q-Learning for 2048 with One-Hot Log Encoding and a Dueling Residual Q-Network

MATH 4900 Final Report

(Bobby Zhang)

December 17, 2025

1 Introduction

This report describes an end-to-end reinforcement learning implementation for the game 2048 using Python/NumPy and PyTorch. The codebase contains a custom 2048 environment with explicit move/merge dynamics and stochastic tile spawning (`Game2048Env.py`), a board encoder that maps the 4×4 grid into a flattened one-hot representation in log space (`one_hot.py`), a dueling Q-network with residual blocks (`dqn_model.py`), and a DQN agent with replay memory, Double-DQN targets, gradient clipping, and soft target-network updates (`dqn_agent.py`). Training is performed by an episodic loop that repeatedly resets the environment, chooses actions via ϵ -greedy control, stores transitions, and updates the Q-network (`train.py`).

For this project, I chose the “write a paper summarizing what I learned” option rather than building an interactive website demo. The main reason is that the current learned policy is not yet reliable enough for a polished user-facing demo (in particular, episode outcomes are sensitive to invalid moves and the maximum tile is still unstable). A website-style demo remains a natural extension of this work and is included as future work.

The goal of this MATH 4900 final project is to present a technically accurate account of what is implemented, what the agent learns under the current design, and why performance plateaus. In my current training outcome (as reflected by the `BestTile` statistic computed in `train.py`), the learned policy reaches a maximum tile that fluctuates around 512. The remainder of the report specifies the MDP, the reward shaping, the state encoding, the network architecture, and the optimization procedure exactly as implemented, and then analyzes the observed plateau using the code-level design choices that shape the learning dynamics.

2 Problem Formulation

The implementation treats 2048 as a Markov decision process with a discrete action space. The state is a 4×4 integer grid $B \in \{0, 2, 4, \dots\}^{4 \times 4}$. The action is an integer $a \in \{0, 1, 2, 3\}$ mapped to the four move directions by the `Action` enum (`UP`, `DOWN`, `LEFT`, `RIGHT`). The transition applies deterministic move-and-merge dynamics and then introduces stochasticity by spawning a new tile in an empty cell. In the environment implementation, the spawned tile is 2 with probability 0.9 and 4 with probability 0.1, and the location is sampled uniformly from empty cells.

The episode terminates if a tile of value at least 2048 exists or if the board is full and no merges are possible. In addition, the environment terminates immediately when the agent selects an invalid action that does not change the board. This additional termination rule materially changes exploration because invalid actions become catastrophic rather than merely unproductive.

3 Environment Dynamics and Reward

The environment implements all four move directions by rotating the board so that a single “move-left” routine can be reused. Concretely, `_move` maps each direction to a rotation count, rotates the board with `np.rot90`, applies `_move_left`, and rotates back. The `_move_left` routine compresses each row, merges adjacent equal tiles at most once per move, and computes a traditional merge score equal to the sum of newly formed tiles in that move.

The top-level `step` function does not directly use this merge score in the returned reward. Instead, it first determines whether the chosen action produces any change to the board and assigns a unit survival reward, setting `reward = 1` if the board changes and `reward = 0` otherwise. In addition, a shaped reward term is defined based on a potential function measuring the squared mass of the board,

$$V(B) = \sum_{i=1}^4 \sum_{j=1}^4 B_{ij}^2.$$

The environment maintains a scalar variable `board_value` storing the value of this potential from the previous step.

In an earlier version of the implementation, the potential baseline `board_value` was updated immediately after the move-and-merge operation and before stochastic tile spawning. As a result, the baseline corresponded to a pre-spawn board, whereas the state returned to the agent already included a newly spawned tile. This ordering introduced a subtle misalignment: part of the measured increase in $V(B)$ at the next step could be attributed to tiles spawned in the previous transition rather than to the causal effect of the current action.

During recent inspection of the environment, this misalignment was identified and corrected. In the current implementation, tile spawning is performed before updating the potential baseline, and the change in $V(B)$ is computed using the post-spawn board state that is actually returned to the agent. For a valid action, the resulting reward takes the form

$$r_t = 1 + \mathbf{1}[V(B_t) > v_{t-1}] \cdot \log_2(V(B_t) - v_{t-1}),$$

where B_t denotes the post-spawn board state and v_{t-1} is the stored potential from the previous step. This modification ensures that changes in the shaping potential are causally aligned with the observed state, reducing noise in credit assignment.

Invalid actions are treated as terminal failures. If the chosen action does not modify the board, the environment immediately terminates the episode and returns reward 0 with `done=True`.

4 State Representation

The training loop encodes the board using `one_hot_encode(board)`. The encoder constructs a tensor of shape $4 \times 4 \times 12$ and flattens it into a 192-dimensional vector. For each tile value $v > 0$ at position (i, j) , it sets channel index $k = \text{int}(\log_2(v))$ to 1.0; empty tiles produce all zeros across channels. The result is a categorical representation of tile magnitudes in log space, without an explicit “empty” channel.

5 Q-Network Architecture

The Q-function approximator is the `DQN` class in `dqn_model.py`. It first projects the 192-dimensional input to a hidden dimension of 64 using a linear layer. It then applies three residual blocks, each

consisting of LayerNorm, a linear transform, a tanh nonlinearity, and a second linear transform, with a residual connection:

$$x \leftarrow x + W_2 \tanh(W_1 \text{LN}(x)).$$

After the residual stack, the network uses a dueling decomposition with separate linear heads for advantage and value. The advantage head outputs $A(s) \in \mathbb{R}^4$ and the value head outputs $V(s) \in \mathbb{R}$. The network returns

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{4} \sum_{a'} A(s, a') \right).$$

6 Learning Algorithm and Optimization

The agent is implemented in `dqn_agent.py`. It maintains an online network (`policy_net`) and a target network (`target_net`), initialized to identical weights. It stores transitions $(s, a, r, s', \text{done})$ in a replay buffer implemented as a bounded deque and begins gradient updates once the buffer contains at least one batch.

Action selection is ϵ -greedy with an exponential decay schedule computed in `_eps_threshold`. With probability $\epsilon(t)$ the agent samples uniformly from $\{0, 1, 2, 3\}$; otherwise it chooses $a = \arg \max_a Q_\theta(s, a)$. Although commented code exists for legality masking, the current implementation does not mask invalid actions, which interacts strongly with the environment’s invalid-action termination.

Training uses a Double-DQN target. For each sampled transition, the online network selects the maximizing next action $a^* = \arg \max_a Q_\theta(s', a)$ and the target network evaluates that action. With discount factor γ , the target is

$$y = r + \gamma Q_{\theta^-}(s', a^*) (1 - \text{done}).$$

The loss is Smooth L1 regression plus an explicit quadratic penalty on predicted Q-values:

$$\mathcal{L}(\theta) = \text{Huber}(Q_\theta(s, a), y) + 0.01 \mathbb{E}[Q_\theta(s, a)^2].$$

Optimization uses Adam, gradients are clipped by global norm 1, and the target network is updated by Polyak averaging each training step:

$$\theta^- \leftarrow (1 - \tau)\theta^- + \tau \theta.$$

7 Training and Logged Metrics

Training is driven by `train.py`. Each episode resets the environment, encodes the initial board, and iterates until `done`. At each step it computes and optionally prints Q-values, selects an action via `agent.choose_action`, steps the environment, encodes the next board, stores the transition via `agent.store`, and performs one update via `agent.train_step`. The script accumulates `total_reward` as the sum of shaped rewards and tracks `max_tile` as the maximum board entry observed during the episode. Progress logging reports `AvgScore` (rolling mean of `total_reward`), `BestTile` (rolling maximum of `max_tile`), and the current ϵ value.

8 Results

In current runs, the maximum tile fluctuates around 512 as measured by the per-episode maximum tile tracked in `train.py`. This regime suggests the agent has learned nontrivial merging behavior and can survive for many steps, but it does not yet exhibit the consistent late-game board management required for reliable 1024+ performance.

9 What I Learned

This project made it clear that “getting a DQN to train” is not the same as learning a policy that aligns with the intended objective. In 2048, small reward-design decisions can dominate what the agent learns, because they directly determine which behaviors are reinforced and how the value function scale evolves under bootstrapping. In particular, the survival-plus- $\log_2(\Delta V)$ shaping was sufficient to learn mid-level merging behavior, but it did not reliably induce the positional invariants needed for late-game play, and the timing of the baseline update before tile spawning introduced a measurable mismatch between what the agent observes and what the shaping term uses as a reference.

I also learned that action validity needs to be handled explicitly when the environment treats invalid moves as terminal. Without action masking, ϵ -greedy exploration and even greedy exploitation can terminate episodes prematurely, which reduces the frequency of deeper states in the replay buffer and amplifies variance. In hindsight, either masking invalid moves or changing the environment so invalid moves are non-terminal is not merely a refinement; it changes the effective data distribution that the value function is trained on.

Finally, I learned why stability tricks matter in practice but are not a complete solution. Double-DQN targets, a dueling head, soft target updates, and gradient clipping made training numerically stable enough to progress, but they did not remove the underlying long-horizon difficulty of 2048, where a large part of performance depends on planning under stochastic tile spawns. This supports the conclusion that further gains likely require either improved shaping aligned with structural board control, or a hybrid approach that combines learning with shallow search.

10 Future Work

A direct improvement is to incorporate stronger spatial inductive bias into the Q-function approximator. Since effective 2048 play depends heavily on board geometry (corner control, monotone gradients, and empty-space management), replacing the flattened MLP with a small convolutional encoder (or at least adding convolutional layers before the dueling head) would allow the network to represent these patterns more efficiently and may improve sample efficiency beyond the current plateau. A second improvement is to redesign reward shaping so that any potential-like term is computed consistently on the post-transition board that is returned to the agent, avoiding the current pre-spawn baseline mismatch. A third direction is to improve long-horizon learning by incorporating multi-step targets or prioritized replay. Beyond purely model-free methods, 2048 is well suited to hybrid approaches that combine a learned value function with shallow search (e.g., expectimax-style lookahead), which is a plausible route to surpassing the current plateau.

11 Hyperparameters (as Implemented)

1. Environment parameters in `Game2048Env.py`: board size 4×4 ; spawn probabilities $P(2) = 0.9$ and $P(4) = 0.1$; terminal if any tile ≥ 2048 or board full with no adjacent equal tiles; episode also terminates immediately on invalid action.
2. Reward parameters in `Game2048Env.step`: survival reward 1 for valid moves and 0 for invalid; shaping potential $V(B) = \sum B_{ij}^2$; additional term $\log_2(V(B_t) - v_{t-1})$ when V increases; baseline `board_value` updated before tile spawning.
3. State encoding in `one_hot.py`: tensor shape $(4, 4, 12)$ with channel index $k = \log_2(v)$ for tile value $v > 0$; flattened input dimension 192; no explicit empty channel.
4. Network parameters in `dqn_model.py`: input dimension 192; hidden dimension 64; residual blocks 3; activation tanh; dueling heads with 4-action advantage and scalar value.
5. Agent parameters in `dqn_agent.py`: number of actions 4; discount factor $\gamma = 0.99$; learning rate 10^{-4} ; replay capacity 50,000; batch size 64; optimizer Adam; loss SmoothL1 plus Q-penalty coefficient 0.01; gradient clip norm 1; soft target update rate $\tau = 5 \times 10^{-3}$.
6. Exploration parameters in `dqn_agent.py`: $\epsilon_{\text{start}} = 0.125$; $\epsilon_{\text{end}} = 0.01$; decay steps 20,000 with exponential schedule $\epsilon(t) = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) \exp(-t/20000)$.
7. Training-loop defaults in `train.py`: `num_episodes=10000`; `log_every=50`; primary logged metrics are rolling `AvgScore`, rolling `BestTile`, and current ϵ .